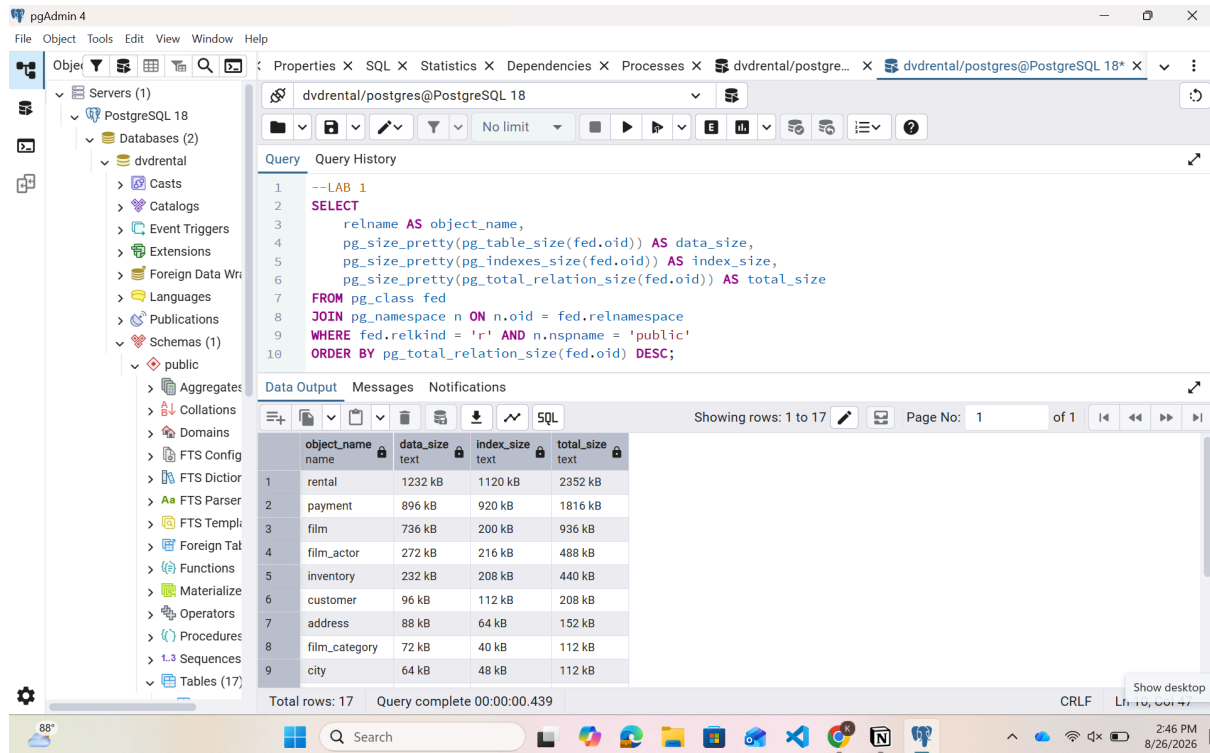


Student name: Nguyễn Khánh Ly

Student ID: 11247193

Class: DS66B

LAB 1



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show the 'public' schema. The 'Query' tab is active, displaying a SQL query that selects table and index sizes for the 'rental' database. The 'Data Output' tab shows the results of the query, which are displayed in a table with 9 rows and 4 columns: object_name, data_size, index_size, and total_size.

	object_name	data_size	index_size	total_size
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	200 kB	936 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB
7	address	88 kB	64 kB	152 kB
8	film_category	72 kB	40 kB	112 kB
9	city	64 kB	48 kB	112 kB

Goal: Identify which tables in dvdrental are “Index Heavy”

The payment table is the most index-heavy table.

Reasoning: It is the only table where the index size (920 kB) is larger than the actual table data size (896 kB).

LAB 2

pgAdmin 4

File Object Tools Edit View Window Help

Properties X SQL X Statistics X Dependencies X Processes X dvdrantal/postgre... X dvdrantal/postgres@PostgreSQL 18* X

dvdrantal/postgres@PostgreSQL 18

Query

```
3      relname AS object_name,
4      pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
5      pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
6      pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
7  FROM  pg_class fed
8  JOIN  pg_namespace n ON n.oid = fed.relnamespace
9  WHERE fed.relkind = 'r' AND n.nspname = 'public'
10 ORDER BY pg_total_relation_size(fed.oid) DESC;

--LAB 2
-- Step 1: Check current size of 'payment' indexes
14 SELECT pg_size_pretty(pg_indexes_size('payment'));
15
-- Step 2: Create a heavy index on payment_date
17 CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
18
-- Step 3: Check size again
20 SELECT pg_size_pretty(pg_indexes_size('payment'));
```

Data Output Messages Notifications

pg_size_pretty
920 kB

Showing rows: 1 to 1 Page No: 1 of 1

Successfully run. Total query runtime: 77 msec. 1 rows affected.

Total rows: 1 Query complete 00:00:00.077 CRLF Ln 13, Col 1

pgAdmin 4

File Object Tools Edit View Window Help

Properties X SQL X Statistics X Dependencies X Processes X dvdrantal/postgre... X dvdrantal/postgres@PostgreSQL 18* X

dvdrantal/postgres@PostgreSQL 18

Query

```
3      relname AS object_name,
4      pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
5      pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
6      pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
7  FROM  pg_class fed
8  JOIN  pg_namespace n ON n.oid = fed.relnamespace
9  WHERE fed.relkind = 'r' AND n.nspname = 'public'
10 ORDER BY pg_total_relation_size(fed.oid) DESC;

--LAB 2
-- Step 1: Check current size of 'payment' indexes
14 SELECT pg_size_pretty(pg_indexes_size('payment'));
15
-- Step 2: Create a heavy index on payment_date
17 CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
18
-- Step 3: Check size again
20 SELECT pg_size_pretty(pg_indexes_size('payment'));
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 117 msec.

Total rows: Query complete 00:00:00.117 CRLF Ln 16, Col 1

The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 connection. The left sidebar displays the database structure, including the 'public' schema. The main pane shows a SQL query with three steps: checking the current index size, creating a new index, and checking the size again. The 'Data Output' tab shows the result of the first and third steps, indicating the index size is 1488 kB. A status message at the bottom confirms the query was successful.

```

3      relname AS object_name,
4      pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
5      pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
6      pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
7  FROM pg_class fed
8  JOIN pg_namespace n ON n.oid = fed.relnamespace
9  WHERE fed.relkind = 'r' AND n.nspname = 'public'
10 ORDER BY pg_total_relation_size(fed.oid) DESC;
11
12 --LAB 2
13 -- Step 1: Check current size of 'payment' indexes
14 SELECT pg_size_pretty(pg_indexes_size('payment'));
15
16 -- Step 2: Create a heavy index on payment_date
17 CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
18
19 -- Step 3: Check size again
20 SELECT pg_size_pretty(pg_indexes_size('payment'));

```

pg_size_pretty
1488 kB

Successfully run. Total query runtime: 46 msec. 1 rows affected.

Discussion: How many Pages (8KB) did this new index consume?

- Initial index size (Step 1): 920 kB
- New index size after creation (Step 3): 1488 kB
- **Calculation:**
 - Size added by the new index: $1488 \text{ kB} - 920 \text{ kB} = 568 \text{ kB}$
 - Convert to 8KB pages: $568 \text{ kB} / 8 \text{ kB} = 71 \text{ pages}$

=> There are 71 pages in this new index consume.

LAB 3

Scenario: Our marketing team only queries "Expensive" films (Replacement cost > 25).

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure: Servers (1) > PostgreSQL 18 > Databases (2) > dvdrental > public > Tables (17). The main query editor shows the following SQL script:

```
14 SELECT pg_size_pretty(pg_indexes_size('payment'));
15
16 -- Step 2: Create a heavy index on payment_date
17 CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
18
19 -- Step 3: Check size again
20 SELECT pg_size_pretty(pg_indexes_size('payment'));
21
22 -- LAB 3
23 -- Create Partial Index
24 CREATE INDEX idx_expensive_films
25 ON film (replacement_cost)
26 WHERE replacement_cost > 25.00;
27
28 -- Find the size of this specific index
29 SELECT relname, pg_size_pretty(pg_relation_size(relid))
30 FROM pg_stat_user_indexes
31 WHERE relname = 'idx_expensive_films';
```

The "Data Output" tab shows the message: "CREATE INDEX" and "Query returned successfully in 56 msec." A green notification bar at the bottom right states: "Query returned successfully in 56 msec."

The screenshot shows the pgAdmin 4 interface with the same SQL script as above. The "Data Output" tab now displays the results of the final query:

pg_size_pretty
216 kB

The "Data Output" tab also shows the message: "SELECT pg_size_pretty(pg_indexes_size('film'));" and "Query returned successfully in 56 msec." A green notification bar at the bottom right states: "Successfully run. Total query runtime: 66 msec. 1 rows affected."

Task: Compare this size with a hypothetical full index on replacement_cost

- Partial Index (idx_expensive_films): Only indexes rows where replacement_cost > 25.00, resulting in a much smaller storage footprint (e.g., fractional size of the table).
- Hypothetical Full Index: Would index all rows in the film table regardless of their replacement cost value.

=> The partial index is significantly smaller in size compared to a full index because it filters out a large portion of rows that do not meet the condition.